

```
import json
import math
import statistics
from collections import Counter, defaultdict
from dataclasses import dataclass
from datetime import datetime
from typing import Any, Dict, List, Optional, Tuple
```

tratamento: Conversão para Float

```
def to_float(value: Any) -> Optional[float]:
    # value: Any: aceita qualquer tipo de valor.
    # -> Optional[float]: pode retornar um float ou None (caso não consiga converter).
    if value is None or value == "":
        return None

    if isinstance(value, (int, float)): # Se já for número (int ou float),
        return float(value)           # apenas converte para float (padroniza o tipo)

    if isinstance(value, str):
        # Se for string, entra no processamento mais interessante
        clean = value.strip().replace("R$", "").replace(".", "").replace(",", ".")
        try:
            return float(clean)
        except ValueError:
            return None

    return None

# teste=to_float("R$ 45,99")
# print(teste, type(teste))

print(to_float("1a"))
print(to_float(""))
print(to_float(10))
print(to_float(3.14))
print(to_float("123"))
print(to_float(" 45.67 "))
print(to_float("1.234,56"))
print(to_float("R$ 99,90"))
print(to_float("abc"))
print(to_float("10,5"))

print()
```

```
None
None
10.0
3.14
123.0
4567.0
1234.56
99.9
None
10.5
```

```
# Recebe qualquer tipo (Any).
# Retorna True, False ou None.
def to_bool(value: Any) -> Optional[bool]:
    if value is None or value == "":
        return None

    if isinstance(value, bool): # Se já for booleano (True ou False), retorna direto.
        return value

    # Converte números: 0 -> False, qualquer outro número -> True
    if isinstance(value, (int, float)):
        return bool(value)

    if isinstance(value, str): # Se for string: remove espaços (strip) e converte
        text = value.strip().lower()

        if text in {"true", "1", "sim", "s", "yes", "y"}:
            return True

        if text in {"false", "0", "nao", "não", "n", "no"}:
            return False

    # Isso facilita comparar valores como "Sim", "SIM" etc.
```

```

    return None

# teste=to_bool("s")
# print(teste, type(teste))

print(to_bool(None))
print(to_bool(""))
print(to_bool(True))
print(to_bool(False))
print(to_bool(0))
print(to_bool(1))
print(to_bool("sim"))
print(to_bool("Não"))
print(to_bool("YES"))
print(to_bool("abc"))

print()

```

```

None
None
True
False
False
True
True
False
True
None

```

```

def parse_date(value: Any) -> Optional[str]:
    # Recebe qualquer tipo (Any)
    # Tenta converter para data padronizada
    if value is None or value == "":
        return None
    if isinstance(value, str):
        for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y", "%Y/%m/%d"):
            # Define lista de formatos aceitos:
            # 2026-04-26 ISO ; 26/04/2026 BR comum ; 2026/04/26 ; 26-04-2026
            try:
                return datetime.strptime(value, fmt).strftime("%Y-%m-%d")
            #.strptime converte string em objeto de data
            # fmt define como a string está formatada
            # strftime converte o objeto de volta para string
            except ValueError:
                continue
        return value
    return None

# teste=parse_date("2026/05/04")
# print(teste)

print(parse_date(None))
print(parse_date(""))
print(parse_date("2026-04-27"))
print(parse_date("27/04/2026"))
print(parse_date("27-04-2026"))
print(parse_date("2026/04/27"))
print(parse_date("04/27/2026"))
print(parse_date("2026.04.27"))
print(parse_date("abc"))
print(parse_date(12345))

print()

```

```

None
None
2026-04-27
2026-04-27
2026-04-27
2026-04-27
04/27/2026
2026.04.27
abc
None

```

```

# Esse código define uma função simples em Python para ler e carregar um arquivo JSON.
# O parâmetro path: str indica que a função espera receber o caminho do arquivo como uma string.
# -> Dict[str, Any] é uma type hint (anotação de tipo), sugerindo que a função retorna um
# dicionário com chaves do tipo str e valores de qualquer tipo (Any).
def load_json(path: str) -> Dict[str, Any]:

```

```

with open(path, 'r', encoding='utf-8') as f:
    return json.load(f)

# Esse código é o complemento natural do anterior: enquanto load_json lê um JSON,
# essa função salva dados em formato JSON em um arquivo.
def save_json(path: str, data: Dict[str, Any]) -> None:
    with open(path, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

# path: str: caminho onde o arquivo será salvo.
# data: Dict[str, Any]: os dados que serão gravados (normalmente um dicionário).
# -> None: indica que a função não retorna nada (apenas executa a escrita).
# json.dump(): converte o objeto Python (data) para JSON e escreve no arquivo f.
# ensure_ascii=False: permite que caracteres como á, ç, à sejam gravados normalmente.
# indent=2: formata o JSON com indentação de 2 espaços, deixando o arquivo mais legível.

```

```

def to_float(v):
    """Função auxiliar para tentar converter valores para float."""
    try:
        return float(v)
    except (ValueError, TypeError):
        return None

```

```

# =====

# Essa função tenta inferir automaticamente o tipo de um campo de dados, ou seja,
# ela olha para uma lista de valores e decide se aquilo parece número, categoria, texto etc.
# Isso é muito usado em análise de dados e sistemas que "auto-entendem" tabelas.
def infer_field_type(values: List[Any]) -> str:
    # Função que tenta descobrir o tipo de uma coluna de dados
    not_null = [v for v in values if v is not None and v != ""] # Remove valores vazios (None e "")
    if not not_null:
        return "indefinido" # Se não sobrou nenhum valor válido

    # Conta quantos valores podem ser interpretados como números
    # (diretamente ou via conversão com to_float)
    numeric = sum(1 for v in not_null if isinstance(v, (int, float)) or to_float(v) is not None)

    if numeric == len(not_null): # Se TODOS os valores são numéricos
        unique = len(set(to_float(v) for v in not_null if to_float(v) is not None))
        # Um set guarda apenas valores únicos (remove duplicatas)
        if unique <= 12: # Poucos valores distintos -> provavelmente discreto
            return "quantitativa_discreta"
        return "quantitativa_continua" # Muitos valores distintos -> contínuo (ex: salário, peso)

    unique = len(set(str(v) for v in not_null)) # Se não for totalmente numérico, trata como texto
    if unique <= 12: # Poucas categorias -> variável categórica simples
        return "qualitativa_nominal"

    return "qualitativa_ordinal_ou_textual" # Muitas categorias -> texto livre ou variável complexa

print(infer_field_type([1, 2, 3, 4]))
print(infer_field_type([1.1, 2.2, 3.3, 4.4]))
print(infer_field_type(list(range(20))))
print(infer_field_type(["1", "2", "3"]))
print(infer_field_type(["a", "b", "c"]))
print(infer_field_type(["a"] * 50))
print(infer_field_type([None, "", None]))
print(infer_field_type(["A", "B", "C", "D", "E", "F"]))
print(infer_field_type([f"texto{i}" for i in range(20)]))
print(infer_field_type([1, "a", 2, "b"]))

print()

quantitativa_discreta
quantitativa_discreta
quantitativa_continua
quantitativa_discreta
qualitativa_nominal
qualitativa_nominal
indefinido
qualitativa_nominal
qualitativa_ordinal_ou_textual
qualitativa_nominal

```

